

PYTHON PROGRAMMING

COMPLETE HANDBOOK — BASIC TO ADVANCED

PART 2

Module 4: Decision Making

Module 5: Loops

Module 6: Strings

Continues from Part 1 (Modules 1-3)

TABLE OF CONTENTS

TABLE OF CONTENTS	2
MODULE 4: DECISION MAKING	4
4.1 if Statement	4
4.2 if-else Statement	4
4.3 if-elif-else Statement	4
4.4 Nested if	5
4.5 Match Case	5
Programs — Module 4	6
Interview Questions — Module 4	6
MCQs — Module 4	7
Exercises — Module 4	7
MODULE 5: LOOPS	8
5.1 for Loop	8
5.2 while Loop	8
5.3 Nested Loops	9
Pattern Programs (Nested Loops)	9
5.4 break	10
5.5 continue	10
5.6 pass	10
Interview Questions — Module 5	11
MCQs — Module 5	11
Exercises — Module 5	12
MODULE 6: STRINGS	13
6.1 String Creation	13
6.2 Indexing	13
6.3 Slicing	13
6.4 String Methods	13
6.5 String Formatting	14
6.6 f-Strings	14
6.7 Escape Characters	15
20+ String Programs	15
Interview Questions — Module 6	16
MCQs — Module 6	17
Practice Questions — Module 6	17
QUICK REVISION NOTES — MODULES 4-6	18

COMMON ERRORS AND FIXES — MODULES 4-6	19
CODING BEST PRACTICES — MODULES 4-6	20

MODULE 4: DECISION MAKING

Decision-making statements let a program choose different actions depending on whether a condition is True or False. Python evaluates the condition and executes only the block that matches.

4.1 if Statement

The simplest decision statement. The indented block runs only if the condition is True; otherwise it is skipped entirely.

```
marks = 85
if marks >= 40:
    print("You passed")
```

Output:

You passed

Flowchart:



4.2 if-else Statement

Provides two paths: one for True, one for False — exactly one of the two blocks always runs.

```
marks = 30
if marks >= 40:
    print("Passed")
else:
    print("Failed")
```

Output:

Failed

4.3 if-elif-else Statement

Used when there are more than two possible outcomes. Python checks each condition top to bottom and runs the first block whose condition is True; if none match, the else block runs.

```
marks = 72
if marks >= 90:
    grade = "A"
elif marks >= 75:
    grade = "B"
elif marks >= 60:
    grade = "C"
else:
    grade = "F"
print("Grade:", grade)
```

Output:

Grade: C

Line	Explanation
2-3	First condition (marks>=90) is False for 72, so this block is skipped.
4-5	Second condition (marks>=75) is also False, skipped.
6-7	Third condition (marks>=60) is True → grade is set to 'C' and Python skips the rest of the chain.
8-9	else is not reached because a matching condition was already found above.

4.4 Nested if

An if statement placed inside another if (or else) block, used when a decision depends on more than one condition in stages.

```
age = 20
has_id = True

if age >= 18:
    if has_id:
        print("Entry allowed")
    else:
        print("ID required")
else:
    print("Too young to enter")
```

Output:

Entry allowed

4.5 Match Case

Introduced in Python 3.10, match-case is Python's version of a switch statement. It compares a value against several patterns and runs the block for the first match.

```

day = 3
match day:
    case 1:
        print("Monday")
    case 2:
        print("Tuesday")
    case 3:
        print("Wednesday")
    case _:
        print("Invalid day")

```

Output:

Wednesday

Note

case `_` is the wildcard pattern — it matches anything, acting like the 'default' case in other languages' switch statements. It should always be placed last.

Programs — Module 4

Program: Check Largest of Three Numbers

```

a, b, c = 12, 45, 7
if a >= b and a >= c:
    largest = a
elif b >= a and b >= c:
    largest = b
else:
    largest = c
print("Largest number is:", largest)

```

Output:

Largest number is: 45

Interview Questions — Module 4**Q1. What is the difference between if-elif-else and multiple separate if statements?**

Ans: In if-elif-else, only ONE matching block runs and Python stops checking further conditions once a match is found. With separate if statements, EVERY condition is checked independently, so more than one block could execute.

Q2. Does Python have a traditional switch statement?

Ans: Not in the classic sense, but Python 3.10+ introduced match-case, which serves a similar purpose using pattern matching.

Q3. What happens if no condition in an if-elif chain is True and there is no else?

Ans: Nothing happens — the program simply moves to the next statement after the chain, since no block was executed.

Q4. What is a nested if?

Ans: An if statement written inside the body of another if or else block, used to check a second condition only after the first is satisfied.

Q5. What does case _ mean in a match statement?

Ans: It is the wildcard/default pattern that matches any value not caught by earlier case blocks.

MCQs — Module 4

No.	Question	Answer	Options (A-D)
1	Which keyword checks multiple conditions in sequence?	C	A) if-if B) switch C) if-elif-else D) while
2	In an if-elif-else chain, how many blocks execute at most?	A	A) One B) All matching C) Two D) Zero always
3	Which Python version introduced match-case?	B	A) 3.6 B) 3.10 C) 2.7 D) 3.0
4	What does case _ represent in match-case?	D	A) Error B) Empty case C) First case D) Default/wildcard case
5	A nested if is:	A	A) An if inside another if B) Two if without else C) Illegal in Python D) Same as elif

Exercises — Module 4

1. Write a program to check whether a year is a leap year.
2. Write a program to categorise a person's BMI using nested if (underweight/normal/overweight).
3. Write a program using match-case to print the name of a month given its number (1-12).
4. Write a program to check if a triangle is valid given three angles (should sum to 180).
5. Write a program to find the largest of four numbers using if-elif-else.

MODULE 5: LOOPS

Loops let a block of code run repeatedly, either a fixed number of times or until a condition becomes False. They avoid writing the same code again and again.

5.1 for Loop

Used to iterate over a sequence (list, string, range, etc.) — it runs once for every item in that sequence.

```
for i in range(1, 6):
    print(i)
```

Output:

```
1
2
3
4
5
```

`range(start, stop, step)` generates numbers from start up to (but not including) stop. `range(5)` alone means 0 to 4.

Dry run of the loop above:

Iteration	i	Condition (i < 6)	Output
1	1	True	1
2	2	True	2
3	3	True	3
4	4	True	4
5	5	True	5
6	6	False	Loop ends

5.2 while Loop

Repeats a block as long as its condition remains True. Used when the number of repetitions is not known in advance.

```
count = 1
while count <= 5:
    print(count)
    count += 1
```


Output:

```
1
2
3
4
5
```

Common Beginner Mistake

Forgetting to update the loop variable (count += 1) inside a while loop creates an infinite loop that never stops — the condition stays True forever.

5.3 Nested Loops

A loop placed inside another loop. The inner loop completes all its iterations for every single iteration of the outer loop.

```
for i in range(1, 4):
    for j in range(1, 3):
        print(f"i={i}, j={j}")
```

Output:

```
i=1, j=1
i=1, j=2
i=2, j=1
i=2, j=2
i=3, j=1
i=3, j=2
```

Pattern Programs (Nested Loops)

Pattern 1: Right-angled triangle of stars

```
rows = 5
for i in range(1, rows + 1):
    print(" " * i)
```

Output:

```
*
**
***
****
*****
```

Pattern 2: Number pyramid

```
rows = 4
for i in range(1, rows + 1):
    print(" " * (rows - i), end="")
    for j in range(1, i + 1):
        print(j, end=" ")
    print()
```

Output:

```

1
1 2
1 2 3
1 2 3 4

```

Pattern 3: Inverted triangle

```

rows = 5
for i in range(rows, 0, -1):
    print("*" * i)

```

Output:

```

*****
****
***
**
*

```

5.4 break

Immediately exits the loop entirely, skipping any remaining iterations, even if the loop condition is still True.

```

for i in range(1, 10):
    if i == 5:
        break
    print(i)

```

Output:

```

1
2
3
4

```

5.5 continue

Skips only the current iteration and moves to the next one; the loop itself keeps running.

```

for i in range(1, 6):
    if i == 3:
        continue
    print(i)

```

Output:

```

1
2
4
5

```

5.6 pass

A null operation — does nothing. Used as a placeholder where Python syntactically requires a statement, such as an empty loop or function body you plan to fill in later.

```
for i in range(5):
    if i == 3:
        pass    # TODO: handle this case later
    print(i)
```

Output:

```
0
1
2
3
4
```

Interview Questions — Module 5

Q1. What is the difference between break and continue?

Ans: break exits the loop completely and no further iterations run. continue skips only the current iteration and moves on to the next one, keeping the loop alive.

Q2. What is the difference between for and while loops?

Ans: A for loop is used when the number of iterations is known or based on a sequence (list, range, string). A while loop is used when repetition depends on a condition and the number of iterations is not fixed in advance.

Q3. What does pass do?

Ans: It is a null statement that does nothing — used as a placeholder to keep code syntactically valid where a statement is required but no action is needed yet.

Q4. What causes an infinite while loop?

Ans: The loop's condition never becomes False, usually because the variable controlling the condition is never updated inside the loop body.

Q5. Can a for loop iterate over a string?

Ans: Yes — strings are sequences of characters, so a for loop can go through each character one at a time, e.g., for ch in 'cat': print(ch).

Q6. What is a nested loop used for?

Ans: To handle two-dimensional repetition, such as printing patterns, working with grids/matrices, or comparing every element of one list with every element of another.

MCQs — Module 5

No.	Question	Answer	Options (A-D)
1	range(5) generates numbers from:	B	A) 1 to 5 B) 0 to 4 C) 1 to 4 D) 0 to 5
2	Which statement exits a loop completely?	A	A) break B) continue C) pass D) exit
3	Which statement skips only the current iteration?	C	A) break B) pass C) continue D) skip

No.	Question	Answer	Options (A-D)
4	A while loop with condition that never turns False causes:	D	A) SyntaxError B) It runs once C) It stops automatically D) Infinite loop
5	pass is used to:	B	A) Exit a loop B) Do nothing / act as placeholder C) Skip iteration D) Print blank line
6	range(2, 10, 2) produces:	A	A) 2,4,6,8 B) 2,3,4...10 C) 2,4,6,8,10 D) 10,8,6,4,2
7	In nested loops, the inner loop:	C	A) Runs once total B) Runs before outer starts C) Completes fully for each outer iteration D) Never runs

Exercises — Module 5

1. Print all even numbers from 1 to 50 using a for loop.
2. Print the multiplication table of a number entered by the user.
3. Print a pyramid pattern of stars, 6 rows tall.
4. Write a program using while loop to reverse the digits of a number.
5. Use break to find and stop at the first number divisible by 7 between 1 and 100.
6. Use continue to print numbers 1-20 while skipping multiples of 3.
7. Write a nested loop program to print a multiplication table (1-5) as a grid.

MODULE 6: STRINGS

A string is a sequence of characters enclosed in single, double, or triple quotes. Strings are one of the most-used data types in Python, from user input to file handling.

6.1 String Creation

```
s1 = 'Hello'
s2 = "World"
s3 = """This is a
multi-line string"""
```

6.2 Indexing

Every character in a string has a position number called an index, starting from 0 for the first character. Negative indexing counts from the end, starting at -1.

```
word = "PYTHON"
print(word[0])    # first character
print(word[-1])   # last character
```

Output:

P
N

P	Y	T	H	O	N	
0	1	2	3	4	5	<- positive index
-6	-5	-4	-3	-2	-1	<- negative index

6.3 Slicing

Slicing extracts a portion of a string using `string[start:stop:step]`. It includes the start index but excludes the stop index.

```
word = "PYTHON"
print(word[1:4])    # index 1 to 3
print(word[:3])     # from start to index 2
print(word[3:])     # from index 3 to end
print(word[::-1])   # reversed string
```

Output:

YTH
PYT
HON
NOHTYP

6.4 String Methods

Method	Purpose	Example	Result
upper()	Converts to uppercase	"hi".upper()	"HI"
lower()	Converts to lowercase	"HI".lower()	"hi"
strip()	Removes leading/trailing spaces	" hi ".strip()	"hi"
replace()	Replaces a substring	"cat".replace("c","b")	"bat"
split()	Splits into a list by delimiter	"a,b,c".split(",")	['a','b','c']
join()	Joins a list into a string	"".join(['a','b'])	"a-b"
find()	Returns index of first match, or -1	"hello".find("l")	2
count()	Counts occurrences	"banana".count("a")	3
startswith()	Checks prefix	"cat".startswith("c")	True
isdigit()	Checks if all characters are digits	"123".isdigit()	True
title()	Capitalises each word	"hi bob".title()	"Hi Bob"
len()	Returns length (built-in function)	len("hello")	5

6.5 String Formatting

Python offers three ways to insert values into strings:

```
name, age = "Neha", 22

# 1. % operator (old style)
print("Name: %s, Age: %d" % (name, age))

# 2. .format() method
print("Name: {}, Age: {}".format(name, age))

# 3. f-string (modern, recommended)
print(f"Name: {name}, Age: {age}")
```

Output:

```
Name: Neha, Age: 22
Name: Neha, Age: 22
Name: Neha, Age: 22
```

6.6 f-Strings

f-strings (formatted string literals) are the modern, preferred way to build strings — prefix the string with `f` and place expressions directly inside `{ }`. They also support inline expressions and formatting specifiers.

```
price = 499.5
print(f"Price: ₹{price:.2f}") # 2 decimal places
print(f"Double: {price * 2}") # expression inside f-string
```

Output:

Price: ₹499.50

Double: 999.0

6.7 Escape Characters

Escape Sequence	Meaning	Example
<code>\n</code>	New line	<code>print("Hi\nBye")</code>
<code>\t</code>	Tab space	<code>print("Hi\tBye")</code>
<code>\\</code>	Backslash	<code>print("C:\\Users")</code>
<code>\'</code>	Single quote	<code>print('It\'s ok')</code>
<code>\"</code>	Double quote	<code>print("She said \"Hi\"")</code>

20+ String Programs

A quick-reference set of short string programs. Type each into your editor and confirm the output yourself.

#	Task	Code	Output
1	Reverse a string	<code>s = "hello"</code> <code>print(s[::-1])</code>	olleh
2	Check palindrome	<code>s = "madam"</code> <code>print(s == s[::-1])</code>	True
3	Count vowels	<code>s = "education"</code> <code>print(sum(1 for c in s if c in "aeiou"))</code>	5
4	Count words	<code>s = "I love Python"</code> <code>print(len(s.split()))</code>	3
5	Convert to uppercase	<code>print("hi".upper())</code>	HI
6	Remove spaces	<code>s = " a b c "</code> <code>print(s.replace(" ", ""))</code>	abc
7	Check substring	<code>print("cat" in "concatenate")</code>	True
8	Concatenate strings	<code>a, b = "Py", "thon"</code> <code>print(a + b)</code>	Python
9	Repeat a string	<code>print("ab" * 3)</code>	ababab
10	Find length	<code>print(len("Python"))</code>	6
11	Convert list to string	<code>lst = ["a", "b", "c"]</code> <code>print("".join(lst))</code>	abc
12	Split sentence into words	<code>s = "I am learning Python"</code> <code>print(s.split())</code>	<code>['I', 'am', 'learning', 'Python']</code>
13	Count occurrence of a char	<code>print("banana".count("a"))</code>	3
14	Check if numeric	<code>print("123".isdigit())</code>	True
15	Capitalise first letter	<code>print("python".capitalize())</code>	Python

#	Task	Code	Output
16	Replace substring	<code>print("I like cats".replace("cats", "dogs"))</code>	I like dogs
17	Check start/end	<code>s = "python.py" print(s.endswith(".py"))</code>	True
18	Swap case	<code>print("PyThOn".swapcase())</code>	pYtHoN
19	Format with f-string	<code>n = "Amit" print(f"Hi {n}!")</code>	Hi Amit!
20	Strip and title-case a name	<code>s = " neha sharma " print(s.strip().title())</code>	Neha Sharma
21	Count characters excluding spaces	<code>s = "a b c" print(len(s.replace(" ", "")))</code>	3
22	Check anagram	<code>a, b = "listen", "silent" print(sorted(a) == sorted(b))</code>	True

Full worked example — Palindrome Checker (with line-by-line):

```
word = input("Enter a word: ")
word = word.lower()
if word == word[::-1]:
    print("Palindrome")
else:
    print("Not a Palindrome")
```

Line	Explanation
1	Take a word as input from the user (returned as a string).
2	Convert to lowercase so the comparison is not case-sensitive (e.g., 'Madam' still matches).
3	<code>word[::-1]</code> reverses the string; compare it to the original.
4-6	Print the appropriate message based on the comparison result.

Output:

```
Enter a word: Madam
Palindrome
```

Interview Questions — Module 6

Q1. Are strings mutable or immutable in Python?

Ans: Immutable. Once created, a string's contents cannot be changed in place — any 'modification' (like `replace()` or concatenation) actually creates and returns a brand-new string object.

Q2. What is the difference between slicing and indexing?

Ans: Indexing accesses a single character using one position, e.g., `s[0]`. Slicing extracts a range of characters using `s[start:stop:step]` and always returns a new string.

Q3. How do you reverse a string in Python?

Ans: Using extended slicing with a step of -1: `s[::-1]`. This is the shortest and most common approach.

Q4. What is the difference between `find()` and `index()`?

Ans: Both search for a substring and return its position. `find()` returns -1 if not found, while `index()` raises a `ValueError` if the substring is not found.

Q5. What are f-strings and why are they preferred?

Ans: f-strings are formatted string literals introduced in Python 3.6 (prefix `f` before the quotes) that let you embed expressions directly inside `{ }`. They are preferred for being more readable and faster than `%` formatting or `.format()`.

Q6. How would you check if a string contains only digits?

Ans: Using the `isdigit()` method, e.g., `'123'.isdigit()` returns `True`, while `'12a'.isdigit()` returns `False`.

MCQs — Module 6

No.	Question	Answer	Options (A-D)
1	Strings in Python are:	B	A) Mutable B) Immutable C) Numeric D) Boolean
2	<code>"Python"[1:4]</code> returns:	C	A) Pyth B) ytho C) yth D) Python
3	Which method converts a string to uppercase?	A	A) <code>upper()</code> B) <code>UPPER()</code> C) <code>toUpper()</code> D) <code>capital()</code>
4	<code>"hello"[:-1]</code> gives:	D	A) hello B) Hello C) HELLO D) olleh
5	Which method splits a string into a list?	B	A) <code>join()</code> B) <code>split()</code> C) <code>divide()</code> D) <code>break()</code>
6	<code>len("Python")</code> returns:	C	A) 5 B) 7 C) 6 D) Error
7	Which is the modern preferred string formatting method?	D	A) <code>%</code> operator B) concatenation C) <code>.format()</code> D) f-strings
8	<code>\n</code> represents:	A	A) New line B) Tab C) Backslash D) Null
9	<code>"cat" + "fish"</code> evaluates to:	B	A) cat fish B) catfish C) Error D) cat+fish
10	Negative indexing starts from:	C	A) 0 B) 1 C) -1 D) -0

Practice Questions — Module 6

- Write a program to count how many times a given character appears in a string.
- Write a program to check whether two strings are anagrams of each other.
- Write a program to remove all vowels from a given string.
- Write a program to convert a sentence into title case without using `title()`.
- Write a program to find the longest word in a sentence.
- Write a program to check whether a string contains only alphabets.
- Write a program that takes a full name and prints the initials (e.g., 'John Ray Smith' -> 'JRS').
- Write a program to replace every space in a sentence with an underscore.

QUICK REVISION NOTES — MODULES 4-6

- if / if-else / if-elif-else choose one path of execution based on conditions checked top-to-bottom; only the first True block runs.
- match-case (Python 3.10+) is Python's pattern-matching alternative to switch; case _ is the default/wildcard.
- for loops iterate over a known sequence (often via range()); while loops repeat until a condition turns False.
- break exits a loop entirely; continue skips only the current iteration; pass does nothing (placeholder).
- Nested loops run the inner loop completely for every single pass of the outer loop — key for patterns and grids.
- Strings are immutable and zero-indexed; s[::-1] reverses a string; slicing excludes the stop index.
- f-strings (f"...{var}...") are the modern, recommended way to format strings in Python 3.6+.

COMMON ERRORS AND FIXES — MODULES 4-6

Error	Cause	Fix
Infinite while loop (program hangs)	Loop variable never updated inside loop	Ensure the condition variable changes each pass
IndentationError inside if/for	Block not indented consistently	Use 4 spaces for every line in the block
TypeError: 'str' object does not support item assignment	Trying to change a character directly, e.g. <code>s[0]='A'</code>	Strings are immutable — build a new string instead
SyntaxError near match	Using match-case on Python version < 3.10	Upgrade Python or use if-elif-else instead
IndexError: string index out of range	Accessing an index beyond the string length	Check <code>len(s)</code> before indexing, or use slicing which never errors

CODING BEST PRACTICES — MODULES 4-6

- Prefer if-elif-else over multiple separate if statements when only one outcome should happen.
- Avoid deeply nested if blocks (more than 2-3 levels) — consider combining conditions with and/or instead.
- Always ensure a while loop's condition variable is updated inside the loop to avoid infinite loops.
- Use for loops with range() when the number of iterations is known; reserve while for condition-driven repetition.
- Prefer f-strings over % or .format() for new code — they are more readable and faster.
- Remember strings are immutable — methods like replace() and upper() return new strings; they don't change the original.